

## UNIT IV

### FILE HANDLING

File handling in Python allows you to create, read, write, and manipulate files stored on your system. It is useful for storing data permanently. Files can be text files (.txt, .csv, .json) or binary files (.jpg, .pdf).

#### OPENING A FILE

Use the built-in open() function:

```
file = open("filename.txt", "mode")
```

#### FILE MODES

| MODE | DESCRIPTION                                     |
|------|-------------------------------------------------|
| "r"  | Read (default, file must exist)                 |
| "w"  | Write (creates new file or overwrites existing) |
| "a"  | Append (adds to end of file)                    |
| "r+" | Read + Write                                    |
| "w+" | Write + Read (overwrites)                       |
| "a+" | Append + Read                                   |
| "b"  | Binary mode (e.g., "rb", "wb")                  |

#### EXAMPLE

```
file = open("example.txt", "r")
```

#### READING FROM A FILE

**METHOD 1:** read() → Reads entire file as a string.

```
content = file.read()
print(content)
```

**METHOD 2:** readline() → Reads one line at a time.

```
line1 = file.readline()
line2 = file.readline()
```

**METHOD 3:** readlines() → Returns a list of all lines.

```
all_lines = file.readlines()
```

#### LOOPING THROUGH LINES (MEMORY EFFICIENT)

```
for line in file:
    print(line)
```

#### EXAMPLE:

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

## WRITING TO A FILE

Write functions are used to store data into a file. Python provides mainly two methods:

- **METHOD 1: write()**

- Writes a single string to a file.
- Accepts only string data
- Returns number of characters written
- Does NOT add a newline automatically
- EXAMPLE:

```
file = open("example.txt", "w")
file.write("Hello, Python!\n")
file.write("Second line.")
```

- **METHOD 2: writelines()**

- Writes multiple strings (from a list or iterable) into a file.
- Takes a list of strings
- Does NOT add newline automatically
- Faster for multiple writes
- EXAMPLE:

```
lines = ["Hello\n", "World\n", "Python\n"]
with open("data.txt", "w") as file:
    file.writelines(lines)
```

## CLOSING A FILE

Always close files to free system resources.

**METHOD 1:**    close()

```
file.close()
```

**METHOD 2:**    with Statement (Recommended) → Automatically closes file.

```
with open("example.txt", "r") as file:
    content = file.read()            (# File closes automatically here )
```

## FILE POSITION OPERATIONS

### 1. GETTING CURRENT POSITION

```
with open('example.txt') as file:
    print(file.tell()) # Current position in file
```

### 2. CHANGING POSITION

```
with open('example.txt') as file:
    file.seek(10) # Move to 10th byte
    print(file.read(5)) # Read next 5 bytes
```

## BEST PRACTICES

1. Always use context managers (with statement) for automatic file closing
2. HANDLE EXCEPTIONS:

```
try:
    with open('file.txt') as f:
        content = f.read()
except FileNotFoundError:
    print("File not found")
except IOError:
    print("Error reading file")
```

3. Use absolute paths for better portability
4. Close files explicitly if not using context managers
5. Be careful with 'w' mode as it overwrites existing files

## WORKING WITH CSV & JSON FILES

### 1- CSV (COMMA-SEPARATED VALUES)

```
import csv
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

### 2- JSON (JAVASCRIPT OBJECT NOTATION)

```
import json
# Reading
with open('data.json', 'r') as file:
    data = json.load(file)

# Writing
with open('data.json', 'w') as file:
    json.dump(data, file, indent=4)
```

### 3- CONFIGURATION FILES

```
import configparser
config = configparser.ConfigParser()
config.read('config.ini')
value = config['SECTION']['key']
```

## FILE POINTER & SEEK OPERATIONS

A file pointer is a cursor that indicates the current position in a file from where data will be read or written.

### seek() Function

- seek() is used to move the file pointer to a specific **position** in a file.
- SYNTAX: file.seek(offset, reference\_point)
  - offset → number of bytes to move
  - reference\_point → position from where movement starts

Reference Points

| Value | Meaning           |
|-------|-------------------|
| 0     | Beginning of file |
| 1     | Current position  |
| 2     | End of file       |

EXAMPLE:

```
with open("data.txt", "r") as file:
```

```
file.seek(5)
```

```
print(file.read())
```

*#Moves pointer to **5th byte**, then reads from there*

### Using seek() with tell()

- tell() returns current pointer position.

```
with open("data.txt", "r") as file:
```

```
print(file.tell()) # 0
```

```
file.read(5)
```

```
print(file.tell()) # 5
```

```
file.seek(0)
```

```
print(file.tell()) # 0
```

### EXAMPLE

File content: "Hello World"

```
with open("data.txt", "r") as file:
```

```
file.seek(6)
```

```
print(file.read())
```

*OUTPUT: World*

## COMMON FILE OPERATIONS

| TASK                 | CODE EXAMPLE                               |
|----------------------|--------------------------------------------|
| Check if file exists | import os; os.path.exists("file.txt")      |
| Delete a file        | import os; os.remove("file.txt")           |
| Rename a file        | import os; os.rename("old.txt", "new.txt") |
| Get file size        | import os; os.path.getsize("file.txt")     |